

Compositional Polymorphism: Resolving the Object Composition Trilemma

R. Scott McKinney
scott@manifold.systems

Abstract

We identify a fundamental trilemma in object-oriented polymorphism: self-call correctness, independent late-bound components, and static typing. No prior model satisfies all three simultaneously. We present *compositional polymorphism*, a composition model that satisfies all three properties. The model is realized as a compiler plugin for Java, supports arbitrary DAG composition with per-interface dispatch roots, introduces composite-scoped encapsulation, and delivers dispatch performance equivalent to standard virtual dispatch.

Contents

1	Introduction	2
2	Overview	3
3	The Model	4
4	Implementation	8
4.1	Component Transformation	8
4.2	Rewriting	9
4.3	Abstract Component Support	9
4.4	@internal Enforcement	9
5	Evaluation	10
5.1	Dispatch Performance	10
5.2	Expressiveness	10
5.3	Case Study: Teaching Assistant	10
5.4	Test Coverage	12
5.5	Limitations	12
5.6	Safety Guarantees	13
6	Discussion	13
6.1	Structural Root Cause	13

6.2	Language Design Implications	13
6.3	Concurrency	14
6.4	Future Work	14
7	Related Work	15
8	Conclusion	17
A	Summary Table	18

1 Introduction

Object-oriented programming promises polymorphic behavioral reuse. Every mechanism it has produced for delivering that promise sacrifices at least one of three properties:

1. **Self-call correctness** — in all compositions, self-calls within a component dispatch through the composite host (*interface-scoped dispatch*; the *Self problem* [2, 6]: a self-call in a component should invoke the composite’s override, not the component’s own implementation)
2. **Independent, late-bound components** — components are independent objects supplied at construction time, not statically inlined, fused, or ordered into a linear chain; a component may simultaneously carry a distinct composite identity per interface
3. **Static typing** — the composition’s interface contracts are enforced at compile time

Prototype-based delegation [2, 6] solves the Self problem and provides late-bound components, but without static type guarantees. Scala traits [7] and Rust trait impls enforce static typing and support polymorphism, but collapse to a single linearized type at link time—real components are gone. Python’s MRO and Ruby’s mixins linearize the chain, forfeiting independent component identity and, in the case of dynamic languages, static typing. Java interfaces with default methods provide static typing but no delegation semantics. Kotlin’s `by` keyword provides forwarding, not delegation—self-calls do not dispatch through the composite.

Even the closest prior models fall short. Kniesel’s type-safe delegation [9] addresses the Self problem under static typing but restricts each component to a single composite identity, failing P2. Ostermann’s delegation layers [10] achieve correct self-call dispatch and static typing, but components must be defined as virtual nested classes of a common enclosing family type—independently defined classes cannot be composed.

These failures are not academic: they are why implementation inheritance remains in widespread use despite decades of criticism [1]. When composition cannot preserve self-call correctness, practitioners reach for inheritance because it does. Compositional polymorphism closes that gap.

This paper presents **compositional polymorphism**, which satisfies all three properties

— and does so at dispatch cost equivalent to standard virtual dispatch, making the model practical for production use. The core contributions are:

- **The Object Composition Trilemma** — we identify and formalize the trilemma (self-call correctness, independent late-bound components, static typing) and establish that no prior model satisfies all three simultaneously.
- **Interface-scoped dispatch** — we introduce this dispatch primitive, in which the composition receiver is determined per interface rather than per component, and show it is the structural change that resolves the trilemma.
- **A formal model** — compositional polymorphism is defined over a standard typed object model with a central dispatch invariant (Proposition 1) and composite-scoped encapsulation (`@internal`).
- **A practical implementation** — a javac plugin for Java 8–26 realizing the model at $O(1)$ dispatch cost, with open-source release and full IDE integration.

2 Overview

Compositional polymorphism introduces two annotations. `@part` marks a class as a reusable component; `@link` marks a field through which one or more interfaces are delegated. A `@part` class implements one or more interfaces; when wired into a composite via a `@link` field, its self-calls for those interfaces dispatch through the composite host rather than through the component itself.

Consider a minimal example:

```
interface Formatter {
    String prefix();
    String format(String msg);
}

@part class LogFormatter implements Formatter {
    public String prefix() { return "INFO"; }
    public String format(String msg) { return "[" + prefix() + "]"
        + msg; }
}

class AppFormatter implements Formatter {
    @link Formatter fmt;
    AppFormatter(Formatter f) { this.fmt = f; }
    @Override public String prefix() { return "APP"; }
}

new AppFormatter(new LogFormatter()).format("started"); // "[APP]
started"
```

When `AppFormatter` is constructed, the compiler wires `LogFormatter`'s composite identity for `Formatter` to `AppFormatter`. A call to `format("started")` dispatches to `LogFormatter.format()`, and the self-call `prefix()` within `format()` dispatches back through `AppFormatter`—invoking `AppFormatter.prefix()`, not `LogFormatter.prefix()`. This is self-call correctness: the composite's override wins, not the component's own implementation.

Because the `@link` field accepts any `Formatter` at construction time, the component is a real runtime object—not fused into the class hierarchy. A different `Formatter` implementation can be substituted without changing `AppFormatter`. This is independent late-bound components.

All of this is enforced at compile time. `@link` fields are `private final`; the compiler rejects any use of `this` in a `@part` class that would allow the component's identity to escape the composite dispatch path. This is static typing.

The formal model in the following section generalizes these ideas to DAG composition, where a single component carries distinct composite identities for multiple interfaces simultaneously.

3 The Model

Motivating example

In the model, `@part` annotates component classes and `@link` annotates delegation fields; both are defined formally in the following section. The following example requires all three trilemma properties to produce the correct output and exposes the limits of every prior model. `WorkerPart` implements both `Logger` and `Processor` and, when composed into `App` via `LoggingWorker`, carries distinct composite identities for each interface—simultaneously rooted at `LoggingWorker` for `Logger` dispatch and at `App` for `Processor` dispatch (Figures 1 and 2).

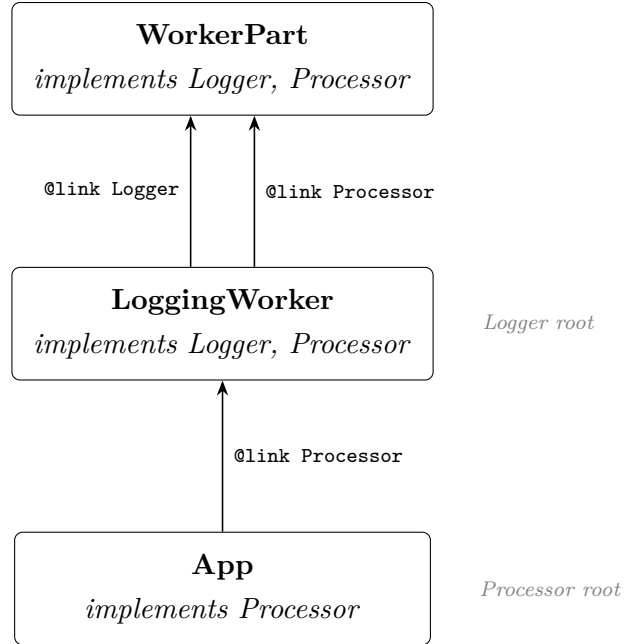


Figure 1: DAG composition structure. Arrows show `@link` fields labeled by the delegated interface. **LoggingWorker** is the root for **Logger** dispatch; **App** is the root for **Processor** dispatch.

```

interface Logger {
    String prefix();
    void log(String msg);
}

interface Processor {
    String label();
    String process(String s);
}

@part class WorkerPart implements Logger, Processor {
    public String prefix() { return "WORKER"; }
    public String label() { return "worker"; }
    public void log(String msg) { System.out.println "[" + prefix()
        + "]" + msg); }
    public String process(String s) {
        log("processing: " + s); // Logger dispatch
        return label() + ":" + s; // Processor dispatch
    }
}

@part class LoggingWorker implements Logger, Processor {
    @link Logger logger;
    @link Processor processor;
    LoggingWorker(WorkerPart w) { this.logger = w; this.processor =
        w; }
    @Override public String prefix() { return "LOGGED"; }
}

class App implements Processor {
    @link Processor worker;
    App(Processor p) { this.worker = p; }
    @Override public String label() { return "APP"; }
}

new App(new LoggingWorker(new WorkerPart())).process("hello");
// Logger dispatch rooted at LoggingWorker -> prefix() ->
// "LOGGED"
// Processor dispatch rooted at App -> label() -> "APP"
// prints: "[LOGGED] processing: hello"
// returns: "APP:hello"

```

Figure 2: DAG composition with multiple composite roots per component. `WorkerPart` carries distinct composite identities for `Logger` (rooted at `LoggingWorker`) and `Processor` (rooted at `App`).

This example is not expressible with any prior model in Table 1. With Kotlin’s `by`, self-calls in `WorkerPart` resolve within `WorkerPart` itself — `prefix()` returns `"WORKER"` and `label()` returns `"worker"` regardless of the composite host, because `by` is forwarding, not delegation. With Scala traits, the example exposes a structural dilemma. To accept a runtime `Processor` as a constructor argument, the Scala programmer must use a concrete object and forward manually—the same forwarding hazard as Kotlin `by`, with the same result: self-calls in `WorkerPart` resolve within `WorkerPart` and `label()` returns `"worker"`, not `"APP"` (P1 fails). The only alternative is to fuse `WorkerPart` via trait inheritance—but then no runtime object can be passed as a parameter at all (P2 fails). There is no Scala arrangement that satisfies both. All three trilemma properties are required to produce the observed output. The following definitions formalize the model.

Definitions

We define compositional polymorphism over a standard typed object model where classes implement interfaces and method dispatch is defined by the receiver’s runtime type.

Definition 1 (Component). *A component is a class annotated `@part`. Components implement one or more interfaces. A component may contain `@link` fields (making it a composite).*

Definition 2 (Link). *A link is a field annotated `@link` in a host class. A link field is `private` and `final`. The delegation surface of a link is the intersection of the field’s declared type and the interfaces of the enclosing class (or the subset specified by `value()`).*

Definition 3 (Composite). *A composite is a class containing one or more `@link` fields. A composite implements the union of the delegation surfaces of all its links.*

Definition 4 (Delegation surface). *The delegation surface of a composite is the set of interfaces delegated by at least one link. Each interface in the surface is assigned to exactly one link; ambiguity (diamond overlap) is a compile error unless resolved with `share=`.*

Definition 5 (Composite identity). *When a component is wired to a composite, it receives a composite identity: a reference to the composite object. Self-calls within the component—calls to methods in the component’s interfaces—dispatch through the composite identity rather than `this`.*

Definition 6 (Interface-scoped dispatch). *Interface-scoped dispatch is the dispatch mechanism by which the composition receiver for a self-call is determined by the interface through which the call is routed, not by the object identity of the caller. Each interface in a component’s delegation surface is assigned an independent composite identity; self-calls route through the composite host for that interface.*

Definition 7 (Transitive composability). *A composite may itself be a component. When a composite-component is wired to a larger composite, composite identity propagates transitively: each interface in the composite-component is re-rooted at the new outermost composite.*

Definition 8 (Delegation surface restriction). *An interface in a component’s `implements` clause may be annotated `@internal` to exclude it from the component’s delegation surface.*

An *@internal* interface participates in the component’s own behavior but cannot be delegated by external composites.

Definition 9 (Composite-scoped method). *An interface method annotated @internal is accessible within the composite family (component-to-component via link fields) but is not accessible through the composite’s external interface. It is the compositional analog of protected without package-scoping.*

Definition 10 (Abstract component). *A component may be declared abstract. Abstract methods in the component are obligations imposed on the delegating composite. The compiler generates a static `asLink()` factory per non-private constructor; `asLink()` returns a deferred instance whose abstract methods throw `DelegationLinkageError` until wired.*

Definition 11 (Diamond resolution). *When two links in a composite share a common interface, exactly one must declare `share=T` for the shared interface. That link is authoritative; the other delegates only its non-shared interfaces. Unresolved diamonds are compile errors.*

Proposition 1. *For any component P in a delegation chain for interface I rooted at composite C : a self-call `this.m()` within P where $m \in I$ dispatches to C . A formal proof in a small-step operational semantics is left to future work (Section 6.4).*

The guarantee holds because wiring is completed before the composite is observable: at any point a self-call can occur, the dispatch slot for I already holds the outermost composite in the chain, so dispatch is correct by construction.

4 Implementation

The implementation is a `javac` plugin operating on the compiler’s AST during two phases. In ENTER, the plugin generates structural scaffolding: the `$selves` composite identity array and `$linkPartToSelf` wiring method for each `@part` class, delegation method skeletons for each composite class, and (for abstract components) the `$Impl` subclass and `asLink()` factories. In ANALYZE, after type resolution, self-calls in `@part` bodies are rewritten, `@link` field assignments are rewritten to invoke wiring, and `@internal` visibility is enforced. No bytecode manipulation is performed; all transformation is at the AST level and subject to standard type checking.

4.1 Component Transformation

Each `@part` class holds a `$selves` array indexed by interface ordinal, assigned at compile time:

```
$selves: Object[]
// initialized: $selves[IDX_I] = this (before wiring)
// after wiring: $selves[IDX_I] = composite identity for
//               interface I
```


An unwired component dispatches on itself; once wired, dispatch routes through the composite identity. `$linkPartToSelf` updates each relevant slot and recurses through any `@link` fields in the component, propagating identity transitively through the DAG. Cycle detection is performed at each slot assignment.

4.2 Rewriting

Within a `@part` method body, every call `this.m(args)` where `m` is declared in interface `I` is rewritten to:

```
((I) $selves[IDX_I]).m(args)
```

In composite classes, each `@link` field assignment `this.link = expr` is rewritten to invoke `$linkPartToSelf`, propagating the composite identity into the component. An $O(1)$ marker interface check guards the call so that non-`@part` assignments pass through unchanged.

Self-preservation. A `@part` class may not reference `this` except as one of its declared interfaces; passing or assigning `this` as the concrete type are compile errors. Field access via `this` is permitted—a part may call back through its own `@link` fields, the compositional analog of `super`. If `this` leaked as the concrete type, a caller could invoke methods outside the composite dispatch path, violating Proposition 1.

4.3 Abstract Component Support

For an abstract `@part` class `Foo`, the compiler generates a private `$Impl` subclass whose abstract method stubs throw `DelegationLinkageError`, and an `asLink()` factory per non-private constructor:

```
private static final class $Impl extends Foo {
    @Override public T m(Params p) {
        throw new DelegationLinkageError("abstract method called
            before linkage");
    }
}
public static Foo asLink(...) { return new $Impl(...); }
```

Once wired, the stubs are unreachable: self-calls dispatch through `$selves[IDX_I]`, which resolves to the composite.

4.4 @internal Enforcement

`@internal` interfaces are excluded from the component’s delegation surface; composite classes cannot expose them via `@link` fields. `@internal` methods are excluded from the composite’s external interface; calls from outside the composite family are compile errors, while calls via `@link` field references within the family are permitted. All enforcement is at compile time with no runtime overhead.

5 Evaluation

5.1 Dispatch Performance

The dispatch path for a self-call through a composite is:

1. Load `$selves[IDX_I]` — array indexed load, compile-time constant index
2. Cast to *I*
3. `invokeinterface m` — standard JVM virtual dispatch

This is functionally $O(1)$. The parallel to vtable dispatch is direct: a vtable call is an indexed slot load followed by an indirect call. In practice, JIT compilers inline trivial forwarders entirely. Composite DAGs are shallow and their call sites are monomorphic or bimorphic in the common case—the JIT’s most aggressively inlined scenarios. Composite dispatch performance is therefore equivalent to direct virtual dispatch in practice.

5.2 Expressiveness

The model supports four composition patterns not achievable together in prior static OO models:

1. **Independent component identity** — components are real objects; their internal state is independent
2. **Interface-scoped dispatch** (Definition 6) — self-calls route through the outermost composite in the delegation chain for the dispatched interface
3. **Multiple composite roots per component** — a component wired for multiple interfaces simultaneously carries distinct composite identities per interface
4. **Composite-scoped encapsulation** — `@internal` provides finer-grained visibility than `public/protected/private` for component families

5.3 Case Study: Teaching Assistant

The following scenario exercises DAG composition, diamond dispatch resolution, and self-call correctness simultaneously. A teaching assistant is both a student and a teacher; both roles share a common `Person` base.

```
interface Person { String getName(); String getTitle(); String
    getTitledName(); }
interface Student extends Person { String getMajor(); }
interface Teacher extends Person { String getDept(); }
interface TA extends Student, Teacher {}
```

Kotlin by. Kotlin’s built-in `by` clause satisfies P2—the implementation is injected at construction time as a real object. P1 does not hold. Delegating `Student` by `s` and `Teacher` by `t` creates a diamond: the compiler forces a manual override of every shared `Person` method. More critically, `getTitledName()`—the template method worth sharing—cannot be safely delegated: whichever branch receives it calls `getTitle()` on its own `this`, bypassing `TaImpl.getTitle()`.

```
class TaImpl(val s: Student, val t: Teacher) : TA,
    Student by s, Teacher by t {
    // Diamond: every shared Person method must be manually resolved
    override fun getName() = s.getName()
    override fun getTitle() = "TA"
    // Without this, s.getTitledName() calls s.getTitle() =
    // "Student", not "TA"
    override fun getTitledName() = "${getTitle()} ${getName()}"
}
```

Every composite in the hierarchy must re-implement `getTitledName()`. Adding a new template method to `Person` requires a matching override in every composite class. `by` is forwarding with cleaner syntax; the composition does not scale.

Compositional polymorphism. `PersonPart` holds the template once. Its self-call to `getTitle()` dispatches through the composite host at each level of the delegation chain.

```
@part class PersonPart implements Person {
    private final String name;
    public PersonPart(String name) { this.name = name; }
    public String getName() { return name; }
    public String getTitle() { return "Person"; }
    public String getTitledName() { return getTitle() + " " +
        getName(); }
}

@part class StudentPart implements Student {
    @link Person person;
    public StudentPart(Person p, String major) { ... }
    public String getTitle() { return "Student"; }
}

@part class TeacherPart implements Teacher {
    @link Person person;
    public TeacherPart(Person p, String dept) { ... }
    public String getTitle() { return "Teacher"; }
}

@part class TaPart implements TA {
    @link(share=Person.class) Student student;
    @link Teacher teacher;
    public TaPart(Student s) {
        this.student = s;
        this.teacher = new TeacherPart(s, "Math");
    }
}
```

```

    }
    public String getTitle() { return "TA"; }
}

TA ta = new TaPart(new StudentPart(new PersonPart("Milton"),
    "CS"));
ta.getTitledName(); // "TA Milton" -- getTitle() dispatches
    through TaPart

```

`getTitledName()` is defined once in `PersonPart` and overridden nowhere. Diamond resolution is handled statically: `@link(share=Person.class)` designates `student` as the authoritative delegate for `Person`, so `teacher` inherits the same `Person` identity rather than introducing a second one. The composition graph is a proper DAG.

Kotlin `by` satisfies P2 but not P1: self-calls in a delegated class dispatch through the delegated object, not the composite host. Compositional polymorphism satisfies all three properties; interface-scoped dispatch (Definition 6) is the mechanism that makes this possible.

5.4 Test Coverage

The implementation is tested against a suite covering:

- forwarding vs. delegation semantics
- DAG composition at depth ≥ 3
- diamond detection and resolution
- abstract component / `asLink()` lifecycle
- `@internal` interface and method enforcement
- default method dispatch
- inheritance chains in `@part` hierarchies
- cycle detection (runtime `DelegationLinkageError`)

5.5 Limitations

- Using `@link`, `@part`, and `@internal` requires configuring the `javac` plugin in the build system (Maven, Gradle, or similar).
- Composite dispatch semantics are not enforced under Java reflection. This is by design.
- The plugin relies on internal `javac` APIs for AST rewriting. These APIs are not part of the official public compiler API and may change across JDK versions. This is a practical constraint of the `javac` plugin approach; it does not affect the model's correctness or generality.

5.6 Safety Guarantees

Cycle detection. If a component is wired to itself directly or transitively, `DelegationLinkageError` is thrown at assignment time during the recursive `$linkPartToSelf()` call.

Deferred method access. If an `asLink()`-produced instance has an abstract method called before wiring, `DelegationLinkageError` is thrown at the stub. This is the compositional analog of `AbstractMethodError`.

@internal violations. All `@internal` enforcement is at compile time; no runtime overhead.

Self-preservation. `@part` classes are not permitted to reference `this` in a context other than a declared interface. This statically prevents component identity from escaping the composite dispatch path, ensuring Proposition 1 holds for all reachable call sites. Field access via `this` is permitted.

6 Discussion

6.1 Structural Root Cause

The design space reveals a progression, borne out by the survey in Section 7. Prior models fall into three categories. *Single-receiver* models—mixins, traits, Kotlin `by`—treat the implementation receiver and the composition receiver as the same object. Mixins and traits satisfy P1 within the structures they support; Kotlin `by` fails P1 outright, routing self-calls within the delegated object rather than the composite host. Multiple inheritance occupies a distinct position: vtable merging preserves P1 (one object, virtual dispatch intact) but destroys independent component identity, failing P2 through object fusion rather than dispatch restriction. *Delegation-based* models—Kniesel’s type-safe delegation, delegation layers, virtual classes—route self-calls through a composite host, satisfying P1, but restrict each component to a single composite identity; independently defined components cannot be composed, failing P2. *Interface-scoped dispatch* (Definition 6)—compositional polymorphism—assigns dispatch identity per interface rather than per object, breaking the assumption that makes P2 intractable. This is the structural change that makes all three properties simultaneously achievable.

6.2 Language Design Implications

The trilemma is not fundamental to statically-typed OO languages per se; it is a consequence of how composition is currently specified and implemented in those languages. Extending any statically-typed OO language with a composition model equivalent to `@link/@part` would suffice.

The result has a direct implication for the long-standing inheritance debate: implementation inheritance is no longer a necessary compromise. Compositional polymorphism directly

addresses each well-documented failure mode:

- **Fragile base class** — components are independent objects; changes to one do not silently affect others.
- **Forced IS-A semantics** — reuse does not require a type relationship; the composite IS-A the interface, not the component.
- **The Self problem** — a hazard introduced when composition replaces inheritance; resolved by routing self-calls through composite roots.
- **Diamond ambiguity** — explicit `share=` resolution; unresolved diamonds are compile errors, not silent merges.
- **Encapsulation breach** — `@internal` provides component-family visibility without exposing internals across package boundaries.
- **Abstract behavior** — abstract components with deferred linkage (`asLink()`) preserve the template-method capability of abstract classes without coupling the hierarchy.

Interface inheritance, which provides type contracts and is foundational to this model, is untouched. The self-preservation constraint—that a `@part` class may not expose `this` as its concrete type—is not a limitation but the static guarantee that makes the dispatch invariant hold; implementation inheritance offers no equivalent assurance.

The Java specifics that make the plugin approach tractable:

- `javac` plugins have full AST read/write access during compilation;
- `invokeinterface` in the JVM supports interface-typed dispatch independently of the concrete class hierarchy;
- interfaces as first-class types allow the `$selves` array to be typed at the interface level.

6.3 Concurrency

The DAG structure is favorable for concurrent use. Component wiring occurs during construction, before the object is shared. Once wired, `$selves` is effectively immutable—no synchronization is required for dispatch. Components may contain mutable state, subject to the usual Java concurrency rules; the dispatch mechanism adds no additional synchronization points.

6.4 Future Work

Intersection type syntax. The natural expression of multi-interface linking—`@link Foo & Bar fooBar;`—would eliminate the `value()` workaround. This requires syntactic extension to the Java parser within the `javac` plugin; feasibility is under investigation.

Formal verification. A formal soundness proof for the dispatch model (Proposition 1) in a small-step operational semantics would strengthen the theoretical foundation. The model maps cleanly to a subset of Featherweight Java; this is planned as follow-on work.

Role-oriented programming. The `@internal` mechanism partially overlaps with role-oriented programming models. A formal comparison is left to future work.

Per-interface composite roots as a general primitive. The structural insight underlying this model—that dispatch identity can be assigned per interface rather than per object—opens a solution space that prior single-composite-identity models cannot reach. Problems currently addressed suboptimally through dynamic dispatch, AOP weaving, or collapsed component hierarchies may admit statically sound solutions within this primitive. This is left as an open research direction.

7 Related Work

Prototype-based delegation. Lieberman [2] introduced the delegation concept: an object may delegate unanswered messages to a prototype, with *self* remaining the original receiver. Self [6] realized this in a working language. These systems solve the Self problem and support late-bound components but provide no static type guarantees and rely on hash-based message dispatch, failing P3.

Behavioral subtyping. Liskov and Wing [3] established the substitutability principle underpinning interface contracts in statically-typed OO. This is a correctness criterion, not a composition mechanism, and so does not appear in the trilemma comparison.

Mixins. Bracha and Cook [4] formalized mixins as parameterized abstract subclasses. Mixins extend through linearization—the mixin is applied to a chain, not composed with independent objects. Self-calls dispatch through `this` in the linearized result, satisfying P1 within the structures mixins support. There are no independent runtime component objects, failing P2.

Traits. Schärli et al. [5] proposed traits as stateless, composable units of behavior. Trait composition is a compile-time operation: methods are flattened into the target class. Self-calls dispatch through `this` in the composed class, satisfying P1 within the structures traits support. There are no independent runtime component objects, failing P2.

Scala traits / abstract type members. Odersky and Zenger [7] introduced statically-typed mixin composition with abstract type members. Scala’s `with` clause linearizes traits into the target class at compile time. Late binding is limited to the linearized type—independent components as runtime objects are not preserved.

In simple compositions where all of a component’s interfaces are delegated together, Scala traits satisfy P1 correctly: self-calls dispatch through the outermost class in the linearized

hierarchy. The limitation is structural and emerges in the general case: a composite cannot hold its own implementation of an interface while selectively delegating a different interface from the same component, because delegation is all-or-nothing. Without selective delegation, per-interface dispatch roots are inexpressible, and P1 fails in that general case. P2 fails entirely—and this is not coincidental. The linearization that enables P1 in the common case requires fusing the component into the class hierarchy; that fusion is precisely what eliminates independent late-bound component objects. In any linearization-based system, satisfying P1 through linearization structurally precludes P2.

Multiple inheritance. Stroustrup [8] introduced virtual base classes and virtual inheritance in C++ to handle diamonds. The result is a single linearized object with a merged vtable. Independent component identity is lost, failing P2.

Kniesel / Lava. Kniesel’s “type-safe delegation” [9] (ECOOP 1999) addresses the Self problem under static typing using immutable delegation links in the Lava language, satisfying P1 within single-rooted compositions and P3. Lava’s delegation model is single-rooted—a component cannot carry independent composite identities for distinct interfaces simultaneously—failing P2.

Delegation Layers. Ostermann’s delegation layers [10] (ECOOP 2002) achieve correct self-call dispatch and enforce type safety, satisfying P1 within single-rooted compositions and P3—but only within a family constraint: components must be defined as virtual nested classes of a common enclosing owner type. Independently defined classes from unrelated hierarchies cannot be composed, failing P2.

Virtual classes and family polymorphism. The virtual-class lineage—gbeta, CaesarJ, Object Teams/Java, and J& [12]—achieves open class composition through virtual nested classes within a common enclosing family type; Ernst, Ostermann, and Cook [11] provide the definitive formal treatment. Self-calls dispatch correctly within the family and static typing is enforced, satisfying P1 within single-rooted compositions and P3. P2 fails across the entire lineage: components are class-level members of an enclosing family type, not independent objects wired at construction time. Introducing a new component combination requires defining a new family subtype—it is class-level inheritance, not runtime object assembly.

Dynamic composition. Newspeak [13] treats classes as first-class late-bound arguments, enabling runtime composition of independent module objects with correct self-call dispatch, satisfying P1 and P2. Static type checking is absent by design, failing P3. This is the strongest known example of a language that deliberately satisfies P1 and P2 at the cost of P3.

Kotlin by-delegation. Kotlin’s `by` keyword [14] generates forwarding methods at compile time. The delegated object’s self-calls resolve within the delegated object—the composite host is invisible. This is forwarding, not delegation, failing P1.

8 Conclusion

We have presented compositional polymorphism, a model that satisfies all three properties of the trilemma — self-call correctness, independent late-bound components, and static typing — realized as a compiler plugin for Java. It operates at compile time via AST rewriting, delivers dispatch performance equivalent to standard virtual dispatch, and supports DAG composition to arbitrary depth, composite-scoped encapsulation, and abstract components with deferred linkage. The key structural contribution is *interface-scoped dispatch*: assigning the composition receiver per interface rather than per component, replacing the single composite identity assumption of prior models. The trilemma is not fundamental to statically-typed OO languages; it is a consequence of that assumption. Any statically-typed OO language extended with an equivalent composition model would satisfy all three properties.

References

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] H. Lieberman. Using prototypical objects to implement shared behavior in object-oriented systems. In *Proceedings of OOPSLA*, pages 214–223, 1986.
- [3] B. Liskov and J. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.
- [4] G. Bracha and W. Cook. Mixin-based inheritance. In *Proceedings of OOPSLA/ECOOP*, pages 303–311, 1990.
- [5] N. Schärli, S. Ducasse, O. Nierstrasz, and A. Black. Traits: Composable units of behaviour. In *Proceedings of ECOOP*, LNCS 2743, pages 248–274, 2003.
- [6] D. Ungar and R. B. Smith. Self: The power of simplicity. In *Proceedings of OOPSLA*, pages 227–242, 1987.
- [7] M. Odersky and M. Zenger. Scalable component abstractions. In *Proceedings of OOPSLA*, pages 41–57, 2005.
- [8] B. Stroustrup. Multiple inheritance for C++. In *Proceedings of the EUUG Spring Conference*, 1987.
- [9] G. Kniesel. Type-safe delegation for run-time component adaptation. In *Proceedings of ECOOP*, LNCS 1628, pages 351–366, 1999. DOI: 10.1007/3-540-48743-3_16.
- [10] K. Ostermann. Dynamically composable collaborations with delegation layers. In *Proceedings of ECOOP*, LNCS 2374, pages 89–110, 2002.
- [11] E. Ernst, K. Ostermann, and W. R. Cook. A virtual class calculus. In *Conference Record of POPL*, pages 270–282, 2006.
- [12] N. Nystrom, X. Qi, and A. C. Myers. J&: Nested intersection for scalable software composition. In *Proceedings of OOPSLA*, pages 347–366, 2006.

- [13] G. Bracha, P. von der Ahé, V. Bykov, Y. Kashai, W. Maddox, and E. Miranda. Modules as objects in Newspeak. In *Proceedings of ECOOP*, LNCS 6183, pages 405–428, 2010.
- [14] JetBrains. Delegation — Kotlin programming language. <https://kotlinlang.org/docs/delegation.html>, 2024.
- [15] R. S. McKinney. manifold-delegation. <https://github.com/manifold-systems/manifold/tree/master/manifold-deps-parent/manifold-delegation>, 2026.

A Summary Table

Model	Self solved	Independent late-bound components	Static typing
Prototype delegation [2, 6]	✓	✓	×
Mixins [4]	✓	×	✓
Traits [5]	✓	×	✓
Kotlin by [14]	×	✓	✓
Kniesel/Lava [9]	✓	×	✓
Delegation Layers [10]	✓	×	✓
Virtual classes [11, 12]	✓	×	✓
Newspeak [13]	✓	✓	×
Compositional polymorphism [15]	✓	✓	✓

Table 1: Comparison of composition models across the three trilemma properties.